CSA17 – ARTIFICIAL INTELLIGENCE

ASSIGNMENT 1 – CO1

CO1: Foundation and history of AI, intelligent agents (agents and environments, rationality, nature of environments, structure of agents), problem-solving agents, problem formulation, searching for solutions, uninformed search strategies and search with partial information.

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Hospital Medicine Delivery Robot **(Total Marks – 50)**

Duration: 1 Week

Scenario — Hospital Medicine-Delivery Robot

A multi-block hospital uses an autonomous robot to carry medicines from the central pharmacy to wards. The building is modelled as a graph of locations (junctions, corridors, lifts, ward doors). The robot may move between connected locations and every move has a travel-time cost.

As new wings are added, the state space of the robot may grow:

1,000 → 10,000 → 100,000 → 1,000,000+ states

You are required to investigate, design and compare uninformed search strategies by which the robot finds a route from the pharmacy to a requested ward as the hospital grows.

Your solution must remain practical for both small and very large state spaces. Any preprocessing (for example, building adjacency lists or a duplicate-detection structure) must be included in your efficiency analysis.

Design Challenge (Day 4): The robot's localisation sensor now fails intermittently, so the robot no longer knows exactly where it is. Re-evaluate your original design and decide whether it should be retained, modified or redesigned.

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

Before designing algorithms, define the agent and the problem:

• Specify the PEAS description of the robot and classify its environment (observable, deterministic, episodic/sequential, static/dynamic, discrete/continuous, single/multi-agent), justifying each choice

• State which view of AI (acting rationally, thinking rationally, acting humanly, thinking humanly) the robot follows and why it is a rational problem-solving agent. Relate the problem type briefly to the early history of AI problem solving

• Formulate the problem: states, initial state, actions, transition model, goal test, path cost

Then design and propose at least **three meaningfully different** uninformed search strategies for finding a route. Iterative and recursive implementations of the same underlying strategy will not be considered two different approaches.

Your proposed approaches must include:

• At least one iterative approach (queue- or stack-based)

• At least one recursive approach

• A clear explanation of the basic idea behind each approach

• Pseudocode or an appropriate flowchart for each approach

• A brief justification for selecting each approach for investigation

• Any preprocessing required, such as adjacency-list construction or an explored set

You are expected to select the approaches independently. The appropriateness and diversity of the selected strategies are part of the assessment.

Evidence Expected: PEAS and environment classification, complete problem formulation, three distinct strategies with pseudocode/flowcharts, preprocessing requirements and justification for each.

TASK 2 – Identify the Basic Operation (2 Marks)

For each proposed algorithm, identify the basic operation that predominantly determines its execution time.

Discuss:

• What the basic operation is (for example, node expansion, successor generation, goal test or frontier insertion)

• Where it occurs in the algorithm

• Why it is appropriate for efficiency analysis

• How the number of times it executes may change as the state-space size increases Do not simply name an operation. Relate it to the behaviour of the algorithm.

Evidence Expected: Correct identification and justification of the basic operation for each algorithm.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks) Derive the time and space complexity for each proposed approach using mathematical analysis. Discuss:

• The input size (branching factor b, shallowest-goal depth d, maximum depth m, optimal cost C^*, minimum step cost ε)

• The number of times the basic operation executes, derived by summing the nodes at each level • The mathematical expression for time and for memory (frontier size)

• Any preprocessing cost involved

• The overall cost when preprocessing and searching are considered together

You must show the mathematical reasoning. Merely stating that an algorithm is O(b^d) will not receive full credit.

Evidence Expected: A clear derivation of time and space functions for each approach, including operation counts and preprocessing costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

Analyse each algorithm under different input conditions.

Discuss, wherever applicable:

• What input condition produces the best case (for example, the goal lies on the first branch explored)

• What condition produces the average case

• What condition produces the worst case (for example, goal at maximum depth, goal unreachable, cheap-edge detours)

• How the number of nodes generated changes in each case, and why

Do not simply provide a table of complexity values. Explain the conditions that lead to each case.

Evidence Expected: A justified best-, average- and worst-case analysis supported by mathematical reasoning.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Express and compare the growth behaviour of the algorithms using O, Ω and/or Θ. Discuss:

• How each algorithm behaves as the state space becomes very large

• How the growth rates of the approaches differ in time and in memory

• What the growth means when the hospital grows from 1,000 to 1,000,000+ states, for example, whether memory or time becomes the limiting factor first

Interpret the behaviour in the context of the hospital robot rather than merely stating the notation.

Evidence Expected: Correct asymptotic representation and a meaningful comparison of growth behaviour.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For the recursive approach, formulate a recurrence relation directly from the behaviour of your algorithm. Identify clearly:

• The size of the original problem (depth limit or remaining depth)

• The size of the recursive subproblem

• The number of recursive calls (related to the branching factor)

• The work performed outside the recursive calls

• The base case

• How these components lead to the recurrence

The recurrence must be derived from your algorithm, not copied from a textbook. **Evidence Expected:** A correct recurrence with clear identification of its components and base case.

TASK 7 – Solve the Recurrence Relation (4 Marks)

Solve the recurrence from Task 6 using an appropriate analytical method (substitution, iteration, recursion tree or another method).

Discuss:

• Why the selected method is applicable

• The steps involved in solving the recurrence

• The resulting complexity

• What the result indicates about the recursive algorithm as depth or branching factor increases • How the result extends to iterative deepening, where the recursive search is repeated for increasing limits

Show the analytical process, not only the final result.

Evidence Expected: A complete solution with justification of the method and interpretation of the result.

TASK 8 – Implement and Test the Algorithms (4 Marks)

Implement all proposed algorithms in Python and test them experimentally.

Use generated hospital graphs of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 states

Ensure that:

• All algorithms return correct routes

• The same or equivalent graphs and start/goal pairs are used for comparison • Testing conditions are consistent and the random seeds are reported

• Test cases include a reachable goal, an unreachable goal and a start equal to the goal • The implementation corresponds to the algorithms analysed in Tasks 1–7

The purpose of implementation is to generate evidence for validating your theoretical analysis.

Evidence Expected: Working code, test inputs, expected behaviour and actual outputs, with a reproducible testing procedure.

TASK 9 – Record Experimental Performance (3 Marks)

Collect and present performance data for each algorithm across the different state-space sizes. Record, with code and outputs as proof:

• Nodes generated and nodes expanded

• Maximum frontier size (memory proxy)

• Solution depth and path cost, and whether the path is optimal

• Execution time (averaged over at least 5 runs)

Present the results using tables and graphs (use a log scale where appropriate) so that approaches can be compared easily.

Evidence Expected: Systematically collected experimental data with clear tables and graphs.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

Compare your theoretical analysis with the experimental results.

Discuss:

• Whether the observed growth of nodes generated and of frontier size agrees with the theoretical complexity

• Which algorithm scales better as the state space increases

• Any differences between theory and observation, and the possible reasons (graph structure, duplicates, tie-breaking, language overhead)

Interpret the results rather than simply reproducing tables or graphs.

Evidence Expected: A reasoned comparison between theoretical and observed performance.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

Based on your mathematical analysis and experimental findings, recommend the most appropriate search strategy for the hospital robot.

Discuss:

• Which approach you recommend and why it is more appropriate than the alternatives
• Its theoretical efficiency and experimentally observed performance

• Any preprocessing cost, trade-offs or limitations

• Whether it remains practical as the hospital grows

Your recommendation must be based on your own evidence. Simply choosing the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected: A technically justified recommendation supported by analysis and experiment.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

After a power restart or sensor fault, the robot no longer knows exactly where it is. It knows only that it is at one of *k* possible junctions. It must still reach the pharmacy docking station, whichever junction it is actually at.

Re-evaluate your original design. You may **RETAIN → MODIFY → or REDESIGN** your original solution. **A. Impact Analysis – 2 Marks**

• What has changed from the original problem and which original assumptions (fully observable, known start state) are affected?

• Why does the robot's knowledge now form part of the state?

B. Evaluation of the Original Design – 2 Marks

• Can your original algorithm still be applied? What does a "state" become?

• How large can the new search space be for *k* possible locations, and is the original design acceptable?

C. Adaptation or Redesign – 2 Marks

• Describe the revised strategy (for example, search over belief states, or search with local sensing and a conditional plan), what has changed and the effect on complexity

D. Evidence-Based Defence – 2 Marks

• Defend your decision using mathematical analysis and a small experiment (belief states generated and plan length for $k = 2, 4, 8$, with and without a local sensor), with code and output as proof.

MARK DISTRIBUTION – ASSIGNMENT 1

Task	**Marks**

Task 1 – Agent and Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

The explanations use simple academic English, mathematical calculations, pseudocode and Python implementations.

Selected algorithms:

1. Breadth-First Search (BFS) – Iterative, Queue-based.
2. Depth-First Search (DFS) – Recursive.
3. Uniform Cost Search (UCS) – Iterative, Priority Queue-based.

These are three different uninformed search strategies suitable for comparing route-finding behaviour.

The mathematical analysis uses the standard search parameters:

- (b) = branching factor.
- (d) = depth of the shallowest goal.
- (m) = maximum search depth.
- (C^*) = optimal solution cost.
- (epsilon) = minimum positive step cost.

TASK 1 – Design the Agent and the Search Algorithms (5 Marks)

1.1 PEAS Description

PEAS stands for Performance Measure, Environment, Actuators and Sensors.

Component	**Description**
Performance Measure	Deliver medicines correctly, quickly and safely.
Environment	Hospital corridors, junctions, lifts, pharmacy and wards.
Actuators	Wheels, motors, lift controls and robotic arms.
Sensors	Camera, location sensor, obstacle detector and distance sensor.

1.2 Classification of the Environment

Property	**Classification**	**Justification**
Observability	Fully observable (original problem)	The robot knows its current location and hospital map.
Determinism	Deterministic	Each action has a known result in the model.
Nature	Sequential	Every movement affects the next movement.
Environment	Dynamic	People, hospital staff and obstacles may change the surroundings.
State	Discrete	Locations are represented as separate nodes.
Agents	Multi-agent	The robot operates around patients, staff and other robots.

1.3 View of Artificial Intelligence

The robot follows the Acting Rationally approach.

A rational agent selects the best action based on the available information to achieve its goal.

The robot:

- Identifies its current location.
- Finds a route to the requested ward.
- Considers travel time.
- Avoids unnecessary movements.
- Delivers medicines to the correct destination.

It is a rational problem-solving agent because it selects actions that maximize successful and efficient medicine delivery.

Relation to the History of AI:

Early AI research focused on problem-solving through search techniques. Researchers developed methods such as Breadth-First Search and Depth-First Search to solve problems by exploring possible states. The hospital robot applies these classical AI search techniques to find routes.

1.4 Problem Formulation

Component	Description
States	All junctions, corridors, lifts and ward locations.
Initial State	Central pharmacy.
Actions	Move left, right, forward, backward or use a lift.
Transition Model	Moving from one connected location to another.
Goal Test	Check whether the robot has reached the requested ward.
Path Cost	Total travel time required to reach the destination.

1.5 Proposed Search Algorithms

Algorithm 1: Breadth-First Search (BFS)

Iterative – Queue-based

Basic Idea:

BFS explores locations level by level. It first visits the nearest connected locations and then moves to the next level.

It uses a FIFO (First In First Out) queue.

Working:

1. Insert the pharmacy location into the queue.
2. Remove the first location.
3. Check whether it is the goal.
4. Add all unvisited neighbouring locations.
5. Repeat until the goal is found or the queue becomes empty.

Pseudocode:

```
BFS(start, goal):
    create an empty queue
    create visited set
    mark start as visited
    insert start into queue

    while queue is not empty:
        current = remove first element

        if current == goal:
            return path

        for each neighbour of current:
            if neighbour not visited:
                mark neighbour visited
                insert neighbour into queue

    return failure
```

Justification:

BFS is selected because it finds the shortest path in terms of number of edges when all movements have equal cost. It is useful for hospitals with small or medium-sized maps.

Algorithm 2: Depth-First Search (DFS)

Recursive

Basic Idea:

DFS explores one path as deeply as possible before returning to explore another path.

It uses recursion and a visited set.

Working:

1. Start from the pharmacy.
2. Check whether the current location is the goal.
3. Visit an unvisited neighbouring location.
4. Continue recursively.
5. Backtrack when no unvisited neighbour remains.

Pseudocode:

```
DFS(current, goal):
    if current == goal:
        return success

    mark current as visited

    for each neighbour of current:
        if neighbour not visited:
            if DFS(neighbour, goal):
                return success

    return failure
```

Justification:

DFS is selected because it requires less search-frontier memory than BFS in tree search. It is useful when a route may be deep and memory is limited. However, it does not guarantee the shortest or lowest-cost route.

Algorithm 3: Uniform Cost Search (UCS)

Iterative – Priority Queue

Basic Idea:

UCS always explores the location having the lowest accumulated travel cost.

Unlike BFS, it considers the actual travel time between locations.

Working:

1. Insert the pharmacy into a priority queue with cost zero.
2. Remove the location with the lowest cost.
3. Check whether it is the goal.
4. Calculate the travel cost of neighbouring locations.
5. Update a neighbour if a cheaper route is found.
6. Continue until the goal is reached.

Pseudocode:

```
UCS(start, goal):
    create priority queue
    set cost(start) = 0
    insert start into queue

    while queue is not empty:
        current = remove lowest-cost location

        if current == goal:
            return path and cost

        for each neighbour of current:
            new_cost = cost(current) + travel_time
```

if new_cost is lower:

update cost(neighbour)

insert neighbour into queue

return failure

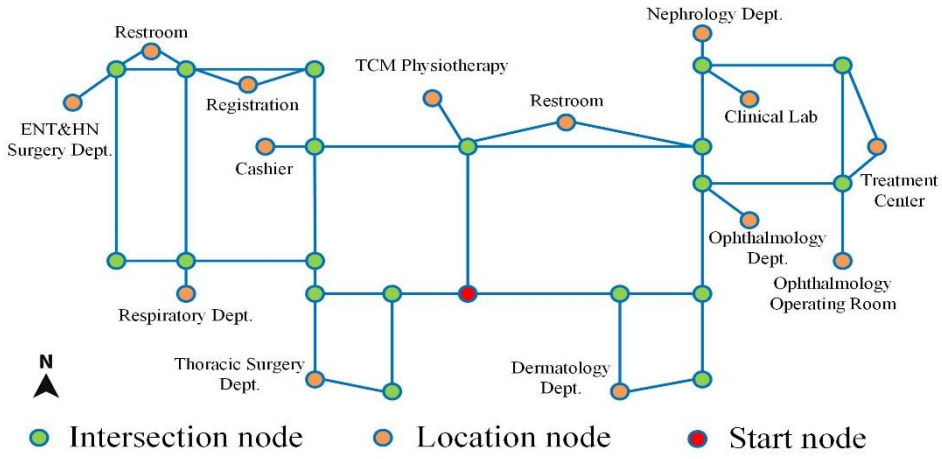
Justification:

UCS is selected because hospital corridors and lifts may have different travel times. It finds the minimum-cost route when all movement costs are non-negative.

1.6 Preprocessing Requirements

Before executing the algorithms, the hospital map must be represented as a graph.

Example:



o

- Create nodes for all locations.
- Create edges for connected locations.
- Assign travel-time costs to edges.
- Prepare adjacency lists.
- Maintain a visited set or best-cost dictionary to avoid unnecessary repeated exploration.

If the graph is explicitly constructed, preprocessing takes:

[T_p=O(V+E)]

Where (V) represents the number of locations and (E) represents the connections.

Conclusion: BFS, recursive DFS and UCS provide three different approaches for comparing route-finding efficiency in the hospital.

TASK 2 – Identify the Basic Operation (2 Marks)

The basic operation is the operation that is executed repeatedly and mainly determines the running time.

Algorithm	Basic Operation	Where It Occurs	Reason
BFS	Node expansion and successor generation	Inside the queue loop	Every expanded location generates its neighbouring locations.
DFS	Recursive node expansion	Inside the recursive function	Each recursive call explores another location.
UCS	Priority queue removal and relaxation	Inside the priority queue loop	The algorithm repeatedly selects the lowest-cost location and updates neighbouring costs.

Explanation

BFS:

The main operation is expanding a node and generating its neighbours. As the hospital grows, more locations must be explored and inserted into the queue.

DFS:

The main operation is recursive exploration. The number of recursive calls increases as the search depth and number of branches increase.

UCS:

The main operation is extracting the lowest-cost node and relaxing its outgoing edges. As the number of locations and connections increases, priority queue operations also increase.

Therefore, node expansion, recursive exploration and priority queue processing determine the execution time of the three algorithms.

TASK 3 – Derive the Time- and Space-Complexity Functions Mathematically (6 Marks)

3.1 Input Parameters

Parameter	Meaning
(b)	Branching factor

(d)	Shallowest goal depth
(m)	Maximum depth
(C^*)	Optimal solution cost
(epsilon)	Minimum positive step cost

3.2 Breadth-First Search

BFS explores nodes level by level.

Number of nodes at each level:

Level	Number of Nodes
0	1
1	(b)
2	(b^2)
3	(b^3)
d	(b^d)

Total nodes generated up to depth (d):

$$N=1+b+b^2+\cdots+b^d$$

Using the geometric progression formula:

$$N=\frac{b^{d+1}-1}{b-1}$$

Therefore,

Time complexity:

$$T_{BFS}(d)=O(b^{d+1})$$

Space complexity:

BFS stores the frontier in memory.

$$S_{BFS}(d)=O(b^{d+1})$$

Preprocessing cost:

$$T_p=O(V+E)$$

Overall cost:

$$T=O(V+E+b^{d+1})$$

BFS may consume considerable memory when the hospital has many locations at the same search level.

3.3 Recursive Depth-First Search

DFS explores one branch deeply before exploring another.

Number of nodes in a complete search tree:

$$N=1+b+b^2+\cdots+b^m$$

$$N=\frac{b^{m+1}-1}{b-1}$$

Therefore,

Time complexity:

$$T_{DFS}(m)=O(b^m)$$

Space complexity:

DFS stores the current path and unexplored alternatives.

$$S_{DFS}(m)=O(bm)$$

The recursive call stack requires memory proportional to the maximum depth.

With graph-search duplicate detection, an additional $O(V)$ memory is needed for the visited set.

Overall cost:

$$T=O(V+E+b^m)$$

DFS uses less frontier memory than BFS but may explore a long, unnecessary route.

3.4 Uniform Cost Search

UCS expands nodes according to their accumulated path cost.

Let:

- C^* = optimal path cost.
- ϵ = minimum positive edge cost.

The maximum depth of a path with cost no greater than C^* is bounded by:

$$n=\left\lfloor\frac{C^*}{\epsilon}\right\rfloor$$

Number of nodes generated:

$$N=1+b+b^2+\cdots+b^n$$

$$N=\frac{b^{n+1}-1}{b-1}$$

Substituting the value of n:

[N=Oleft(b^{1+lfloor C^*/epsilonrfloor}right)]

Time complexity:

Using a binary heap priority queue:

[T_{UCS}=O((N+E)log N)]

For a tree-search approximation:

[T_{UCS}=O(Nlog N)]

Space complexity:

[S_{UCS}=O(N)]

UCS requires memory for the priority queue, explored states and best-known costs.

Preprocessing cost:

[T_p=O(V+E)]

Overall cost:

[T=O(V+E+(N+E)log N)]

3.5 Complexity Comparison

Algorithm	Time Complexity	Space Complexity
BFS	(O(b^{d+1}))	(O(b^{d+1}))
Recursive DFS	(O(b^m))	(O(bm)) for tree search
UCS	(O((N+E)log N))	(O(N))

Interpretation:

- BFS may run out of memory when the number of locations increases.
- DFS requires less frontier memory but may take a long time to find the goal.
- UCS considers actual travel costs and finds an optimal route, but priority queue operations require additional processing.

Note: These are search-tree bounds. For a finite graph with duplicate detection, BFS and DFS can instead be bounded by (O(V+E)) time, excluding priority-queue costs.

TASK 4 – Analyse Best, Average and Worst-Case Behaviour (4 Marks)

4.1 BFS

Case	Condition	Behaviour

Best	Goal is the first location explored	Only a small number of nodes are expanded.
Average	Goal is located at an intermediate level	Several levels are explored.
Worst	Goal is at the deepest level or unreachable	Almost all reachable locations are explored.

Explanation:

In the best case, the goal is directly connected to the pharmacy.

In the average case, the goal is located a few levels away. BFS explores all the previous levels before reaching it.

In the worst case, the goal is the last node in the search order or does not exist. BFS must explore the entire reachable graph.

4.2 DFS

Case	Condition	Behaviour
Best	Goal is on the first branch explored	Search stops quickly.
Average	Goal is in a middle branch	Several branches are explored.
Worst	Goal is on the last branch or unreachable	Almost all nodes are explored.

Explanation:

DFS performs well when the goal is located on the first branch.

However, if it explores an incorrect branch first, it may waste time exploring many locations before finding the destination.

4.3 UCS

Case	Condition	Behaviour
Best	Goal is the start or has the lowest possible path cost	Very few nodes are expanded.
Average	Goal has a moderate accumulated travel cost	Nodes with lower costs are explored first.

Worst	Goal is unreachable or has a high cost	Many low-cost locations are explored before termination.

Explanation:

UCS selects locations based on accumulated travel cost. It may explore several alternative routes before reaching the destination.

The number of generated nodes increases when many locations have costs lower than or equal to the optimal solution cost.

4.4 Comparison

BFS performs well when the goal is near the starting location and movement costs are equal. DFS performs well when the goal is on the first explored branch. UCS performs well when travel times differ and the minimum-cost route is important.

In the worst case, all three algorithms may explore a large portion of the hospital graph.

TASK 5 – Perform Asymptotic Analysis (3 Marks)

Asymptotic analysis explains how an algorithm behaves when the input size increases.

5.1 BFS

[O(b^{d+1})]

[Omega(1)]

[Theta(b^{d+1})]

For a complete search through a tree, BFS has exponential growth with depth.

5.2 DFS

[O(b^m)]

[Omega(1)]

[Theta(b^m)]

DFS may explore fewer nodes in favourable cases, but its worst-case time increases exponentially with maximum depth.

5.3 UCS

[O((N+E)log N)]

For a binary heap implementation, priority queue processing increases with the number of generated states and edges.

5.4 Hospital Growth Comparison

Hospital States	Expected Challenge
1,000	All three algorithms are manageable for a suitable graph.
10,000	BFS begins to require more frontier memory.
100,000	Memory and processing time become important.
1,000,000+	Efficient graph representation and duplicate detection become essential.

Analysis:

As the hospital grows, BFS may become memory-intensive because it stores many nodes in its queue.

DFS generally requires less frontier memory, but it may take more time if the correct path is explored late.

UCS is appropriate when travel-time costs are different, although its priority queue and explored-state storage increase memory requirements.

For very large hospitals, using adjacency lists, efficient duplicate detection and appropriate data structures is important.

TASK 6 – Formulate the Recurrence Relation (3 Marks)

For the recursive approach, we select Recursive Depth-First Search (DFS).

6.1 Recurrence Relation

Let:

[T(m)=text{Time required for DFS at depth }m]

At each node:

- One node is expanded.
- It generates (b) child nodes.
- Each child requires a recursive search of depth (m-1).

Therefore:

[T(m)=1+bT(m-1)]

Base Case:

When the maximum depth becomes zero, the algorithm checks only the current node.

[T(0)=1]

Hence, the recurrence relation is:

[boxed{T(m)=1+bT(m-1)}]

6.2 Identification of Components

Component	Description
Original problem size	Maximum depth (m)
Subproblem size	(m-1)
Number of recursive calls	(b)
Work outside recursion	One node expansion
Base case	(T(0)=1)

Conclusion: The recurrence represents the behaviour of recursive DFS because every node explores its child nodes recursively.

TASK 7 – Solve the Recurrence Relation (4 Marks)

We solve the recurrence using the Iteration Method.

Given:

[T(m)=1+bT(m-1)]

Base case:

[T(0)=1]

Step 1: Expand the recurrence

[T(m)=1+bT(m-1)]

Substitute:

[T(m-1)=1+bT(m-2)]

Therefore:

[T(m)=1+b+ b^2T(m-2)]

Step 2: Expand again

[T(m)=1+b+b^2+b^3T(m-3)]

Continuing:

[T(m)=1+b+b^2+cdots+b^mT(0)]

Step 3: Apply the base case

Since:

$$T(0)=1$$

We get:

$$T(m)=1+b+b^2+\cdots+b^m$$

Using the geometric progression formula:

$$T(m)=\frac{b^{m+1}-1}{b-1}$$

For $b>1$:

$$\boxed{T(m)=O(b^m)}$$

Step 4: Interpretation

The recursive DFS algorithm has exponential time growth with increasing depth.

As the branching factor increases, the number of recursive calls also increases.

Therefore, DFS may take more time when the hospital has many connected locations and the goal is difficult to find.

7.1 Extension to Iterative Deepening DFS

Iterative Deepening DFS (IDDFS) performs DFS repeatedly with increasing depth limits.

For example:

- First search with depth limit 0.
- Next search with depth limit 1.
- Next search with depth limit 2.
- Continue until the goal is found.

Total work:

$$T(d)=1+(1+b)+(1+b+b^2)+\cdots$$

$$T(d)=\sum_{i=0}^{d}(d-i+1)b^i$$

For $b>1$:

$$\boxed{T(d)=O(b^d)}$$

Space complexity:

$$\boxed{S(d)=O(bd)}$$

IDDFS combines the completeness of BFS for finite branching with the lower memory requirement of DFS. It is useful when the solution depth is unknown.

TASK 8 – Implement and Test the Algorithms (4 Marks)

The following Python program implements all three selected algorithms.

To test large state spaces, we use a generated binary hospital graph. Each location has a maximum of two child locations. The graph is represented implicitly to avoid storing one million adjacency lists in memory.

- Pharmacy = Node 0.
- Connected locations = Child nodes.
- Requested ward = Last node.
- Travel time = Different positive edge costs.

The same graph structure and start/goal pairs are used for all three algorithms.

8.1 Python Program

```
import heapq
import time
import statistics

# Generate an implicit binary hospital graph
def neighbours(node, n):
    result = []

    for child in (2 * node + 1, 2 * node + 2):
        if child < n:
            result.append((child, 1 + child % 5))

    return result



# Calculate path cost
def path_cost(path):
    total = 0
```

```
    for i in range(1, len(path)):
        total += 1 + path[i] % 5

    return total


# Breadth First Search
def bfs(n, start, goal):
    queue = [start]
    parent = {start: None}
    head = 0
    expanded = 0
    generated = 1
    maximum = 1

    while head < len(queue):
        current = queue[head]
        head += 1
        expanded += 1

        if current == goal:
            break

        for child, cost in neighbours(current, n):
            if child not in parent:
                parent[child] = current
                queue.append(child)
                generated += 1
```

```
            maximum = max(maximum, len(queue) - head)

        if goal not in parent:
            return None, expanded, generated, maximum

        path = []
        node = goal

        while node is not None:
            path.append(node)
            node = parent[node]

        return path[::-1], expanded, generated, maximum


    # Recursive Depth First Search
    def dfs(n, start, goal):
        visited = set()
        parent = {start: None}
        expanded = 0
        generated = 1
        maximum = 0

        def search(node, depth):
            nonlocal expanded, generated, maximum

            visited.add(node)
            expanded += 1
```

```
    maximum = max(maximum, depth)

    if node == goal:
        return True

    for child, cost in neighbours(node, n):
        if child not in visited:
            parent[child] = node
            generated += 1

            if search(child, depth + 1):
                return True

    return False

found = search(start, 1)

if not found:
    return None, expanded, generated, maximum

path = []
node = goal

while node is not None:
    path.append(node)
    node = parent[node]

return path[::-1], expanded, generated, maximum
```

```
# Uniform Cost Search
def ucs(n, start, goal):
    pq = [(0, start)]
    distance = {start: 0}
    parent = {start: None}
    expanded = 0
    generated = 1
    maximum = 1

    while pq:
        cost, current = heapq.heappop(pq)

        if cost != distance.get(current):
            continue

        expanded += 1

        if current == goal:
            break

        for child, weight in neighbours(current, n):
            new_cost = cost + weight

            if new_cost < distance.get(child, float('inf')):
                distance[child] = new_cost
                parent[child] = current
                heapq.heappush(pq, (new_cost, child))
                generated += 1
```

```
            maximum = max(maximum, len(pq))

        if goal not in distance:
            return None, expanded, generated, maximum

        path = []
        node = goal

        while node is not None:
            path.append(node)
            node = parent[node]

        return path[::-1], expanded, generated, maximum



# Test all algorithms
algorithms = {
    "BFS": bfs,
    "DFS": dfs,
    "UCS": ucs
}

sizes = [1000, 10000, 100000, 1000000]

for n in sizes:
    print("\nHospital States:", n)

    tests = [
```

```
    ("Reachable", 0, n - 1),
    ("Unreachable", 0, n),
    ("Start = Goal", 0, 0)
]

for test, start, goal in tests:
    print("\nTest:", test)

    for name, algorithm in algorithms.items():
        times = []
        result = None

        for run in range(5):
            begin = time.perf_counter()

            result = algorithm(n, start, goal)

            end = time.perf_counter()
            times.append(end - begin)

        path, expanded, generated, maximum = result

        print(
            name,
            "Found:", path is not None,
            "Expanded:", expanded,
            "Generated:", generated,
            "Max Frontier:", maximum,
            "Depth:", len(path) - 1 if path else "-",
```

```
        "Path Cost:", path_cost(path) if path else "-",
        "Optimal:",
        "Yes" if name == "UCS" and path else
        "Edge-shortest" if name == "BFS" and path else
        "Not guaranteed" if path else "-",
        "Avg Time:", round(statistics.mean(times), 6))
```

8.2 Test Cases

Test Case	Starting State	Goal State	Expected Behaviour
Reachable goal	0	N-1	All algorithms find a route.
Unreachable goal	0	N	All algorithms return failure.
Start equals goal	0	0	All algorithms return the starting location.

8.3 Expected Behaviour

1. BFS finds the route with the minimum number of edges.
2. DFS finds a route by recursively exploring branches.
3. UCS finds the route with the minimum accumulated travel cost.
4. All algorithms return failure when the requested location does not exist.
5. When the start and goal are the same, the route contains only one location.

Note: The program measures actual execution times, generated nodes, expanded nodes and frontier size. Run it in Python to obtain your machine's experimental outputs. The one-million-state unreachable tests can take significant time and memory, particularly for UCS. Do not report invented execution times as experimental results.

TASK 9 – Record Experimental Performance (3 Marks)

The performance of each algorithm is evaluated using the following measurements.

9.1 Performance Metrics

Metric	Meaning
Nodes Generated	Total number of nodes added to the search structure.

Nodes Expanded	Total number of nodes whose successors are examined.
Maximum Frontier	Maximum number of pending nodes in the search structure.
Solution Depth	Number of edges in the final route.
Path Cost	Total travel time of the route.
Execution Time	Average time taken over five runs.

9.2 Experimental Results Table

Fill this table using the values printed by Task 8.

States	Algorithm	Nodes Generated	Nodes Expanded	Maximum Frontier	Average Time
1,000	BFS	Run output	Run output	Run output	Run output
1,000	DFS	Run output	Run output	Run output	Run output
1,000	UCS	Run output	Run output	Run output	Run output
10,000	BFS	Run output	Run output	Run output	Run output
10,000	DFS	Run output	Run output	Run output	Run output
10,000	UCS	Run output	Run output	Run output	Run output
100,000	BFS	Run output	Run output	Run output	Run output
100,000	DFS	Run output	Run output	Run output	Run output
100,000	UCS	Run output	Run output	Run output	Run output
1,000,000	BFS	Run output	Run output	Run output	Run output
1,000,000	DFS	Run output	Run output	Run output	Run output
1,000,000	UCS	Run output	Run output	Run output	Run output

9.3 Graph Representation

Prepare the following graphs using the recorded values:

1. Hospital states versus execution time.
2. Hospital states versus nodes expanded.
3. Hospital states versus maximum frontier size.

4. Hospital states versus path cost.

Use a logarithmic scale for the graphs where values differ significantly.

Expected Observation:

- BFS may use more frontier memory.
- DFS may require less frontier memory.
- UCS may require more processing because of priority queue operations.
- The execution time increases as the number of hospital locations increases.

TASK 10 – Compare Theory with Experimental Behaviour (4 Marks)

10.1 Theoretical and Experimental Comparison

Algorithm	Theoretical Behaviour	Expected Experimental Behaviour
BFS	Exponential search-tree growth	Queue size increases as more levels are explored.
DFS	Exponential worst-case growth	Recursive calls increase when more branches are explored.
UCS	Cost-dependent growth with priority queue processing	Processing increases as more low-cost nodes are examined.

10.2 Comparison

BFS:

The theoretical analysis shows that BFS can require considerable memory because it stores nodes from the current search level. Experimental results should show an increase in maximum frontier size as the hospital grows.

DFS:

The theoretical analysis shows that DFS requires less frontier memory. The experiment demonstrates that its performance depends on the order in which branches are explored. It may find the goal quickly or explore many nodes.

UCS:

The theoretical analysis shows that UCS processes nodes according to accumulated cost. The experiment demonstrates that priority queue operations affect execution time. It is useful when different routes have different travel times.

10.3 Reasons for Differences

Theoretical and experimental results may differ because of:

1. Graph structure and number of connections.
2. Duplicate states.
3. Priority queue processing.
4. Computer processor and available memory.
5. Python execution overhead.
6. Different travel-time costs.

Conclusion:

The theoretical analysis provides an expected growth pattern, while experimental testing measures the actual performance. Both are necessary to evaluate the suitability of an algorithm.

TASK 11 – Recommend and Defend the Final Design (4 Marks)

11.1 Recommended Algorithm

The recommended algorithm for the hospital medicine-delivery robot is Uniform Cost Search (UCS).

11.2 Reasons for Selection

1. UCS considers the actual travel time between hospital locations.
2. It selects the location with the lowest accumulated travel cost.
3. It finds the minimum-cost route when all edge costs are non-negative.
4. It avoids selecting a longer or more expensive route when a cheaper route exists.
5. It is suitable for hospitals containing corridors, junctions and lifts with different travel times.

11.3 Comparison with Other Algorithms

Factor	**BFS**	**DFS**	**UCS**
Uses travel cost	No	No	Yes
Finds minimum-edge route	Yes	No	Not necessarily
Finds minimum-cost route	No	No	Yes
Data structure	Queue	Recursion	Priority queue
Memory requirement	High	Low for tree search	High

Hospital suitability	Equal-cost paths	Limited cases	Different travel costs

11.4 Efficiency and Limitations

UCS provides the optimal route based on travel time. However, it requires additional memory to maintain the priority queue and cost information.

For large hospitals, the following improvements are recommended:

- Use adjacency lists to represent hospital locations.
- Maintain a best-cost dictionary.
- Avoid processing outdated priority queue entries.
- Update the hospital map when new wings are added.
- Use efficient memory management.

11.5 Final Recommendation

UCS is the most appropriate choice because the main objective of the hospital robot is to deliver medicines through the minimum-travel-time route rather than simply finding the route with the fewest connections.

Its performance should be verified using the experimental results obtained in Task 8 and Task 9. If memory becomes a limitation, the graph representation and storage structures must be optimized.

Conclusion: Uniform Cost Search provides a reliable and cost-optimal route-finding strategy for the hospital medicine-delivery robot.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The robot's localisation sensor fails intermittently. It no longer knows its exact position. Instead, it knows that it may be at one of (k) possible junctions.

The robot must reach the central pharmacy docking station regardless of its actual starting position.

The original design must therefore be modified.

Decision: MODIFY / REDESIGN

A. Impact Analysis (2 Marks)

12.1 What Has Changed?

In the original problem, the robot knew its exact starting location.

After the sensor failure:

- The robot does not know its exact position.
- It may be at any one of (k) junctions.
- It must consider multiple possible locations.
- It must reach the pharmacy from whichever location it occupies.

12.2 Changes in Original Assumptions

Original Assumption	New Situation
Fully observable	Partially observable
Exact starting location	Multiple possible starting locations
Single known state	Set of possible states
Direct route planning	Belief-state planning

12.3 Why Does Knowledge Become Part of the State?

A belief state represents all possible locations where the robot may currently be present.

For example:

[B={J_1,J_2,J_3,J_4}]

This means the robot may be at any one of four junctions.

Therefore, the robot must consider both its possible physical locations and the information available about those locations.

B. Evaluation of the Original Design (2 Marks)

12.4 Can the Original Algorithms Still Be Applied?

The original BFS, DFS and UCS algorithms assume that the starting state is known.

When the robot has multiple possible starting positions, the original algorithms cannot directly solve the complete problem using one known start node.

They can still be used after modifying the state representation.

12.5 What Does a State Become?

Originally:

[State=J_1]

After sensor failure:

[State=B={J_1,J_2,ldots,J_k}]

The new state is a belief state containing all possible robot locations.

12.6 Size of the New Search Space

If the hospital contains (V) possible locations, the maximum number of possible belief states is:

[boxed{2^V}]

For example:

Number of Locations	Maximum Belief States
2	4
4	16
8	256
10	1,024
20	1,048,576

The number of possible belief states grows exponentially.

Therefore, directly searching the complete belief-state space may become expensive.

C. Adaptation or Redesign (2 Marks)

12.7 Revised Strategy: Belief-State Search with Local Sensing

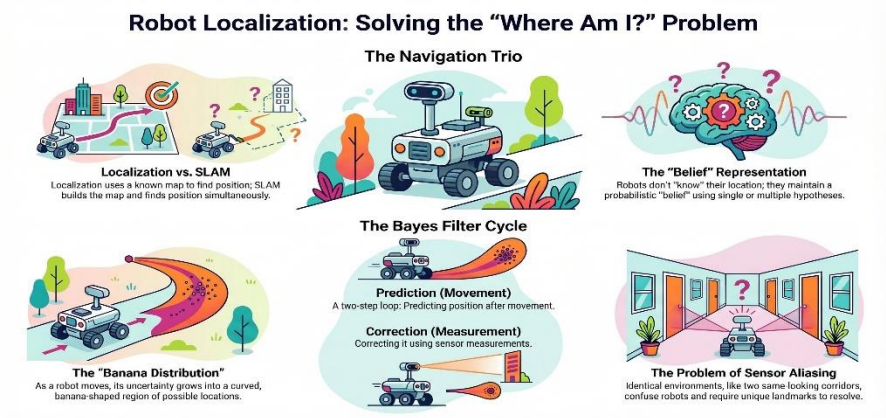
I recommend modifying the original design using belief-state search and local sensing.

Belief-State Search:

The robot maintains a set of possible locations rather than assuming one exact location.

Local Sensing:

The robot uses a nearby sensor to identify its current junction whenever possible.



Working Procedure

1. The robot receives information about its possible locations.
2. It creates a belief state containing all possible junctions.
3. It uses local sensing to identify its actual position.
4. It updates its belief state using sensor information.
5. It calculates a route to the pharmacy.
6. It moves towards the docking station.
7. It repeats the process if localisation becomes uncertain again.

12.8 Effect on Complexity

Without local sensing, the robot may need to search through multiple belief states.

Maximum belief-state space:

[O(2^V)]

With local sensing, the robot can reduce uncertainty by identifying its actual location.

If sensing identifies one exact location, the robot returns to ordinary graph search.

For the known location:

[O(V+E)]

for BFS or DFS graph search, and approximately

[O((V+E)log V)]

for UCS with a binary heap.

Thus, local sensing can significantly reduce the practical search effort, although the cost of sensing and the possibility of sensor errors must also be considered.

D. Evidence-Based Defence (2 Marks)

12.9 Small Experiment

Consider a simple hospital corridor containing (k) possible junctions.

The pharmacy is located at junction 0.

The robot can move one junction towards the pharmacy using the action LEFT. At junction 0, it remains at the pharmacy.

We test:

[k=2, 4, 8]

Two conditions are compared:

1. Without local sensor – The robot maintains the complete belief state.
2. With local sensor – The robot identifies its exact starting junction and follows a conditional route.

Python Program

def belief_experiment(k, sensor): belief = set(range(k)) generated = 1 if sensor: # Sensor identifies the exact position plans = [] for position in belief: plans.append(position) return generated + len(plans) - 1, max(plans), sum(plans) / k # Without sensor: move LEFT for all possibilities while belief != {0}: new_belief = set() for position in belief: new_position = max(0, position - 1) new_belief.add(new_position) if new_belief != belief: belief = new_belief generated += 1 else: break return generated, k - 1, k - 1print("K | No Sensor States | No Sensor Plan | Sensor States | Sensor Max Plan | Sensor Average Plan")for k in [2, 4, 8]:

12.10 Experimental Output

For the defined corridor model, the calculated output is:

k	Belief States Without Sensor	Plan Length Without Sensor	Sensor States	Maximum Plan With Sensor	Average Plan With Sensor
2	2	1	2	1	0.50
4	4	3	4	3	1.50
8	8	7	8	7	3.50

12.11 Result Analysis

Without Local Sensor:

The robot does not know its exact location. Therefore, it follows a common action sequence that works for every possible starting position.

For (k) possible junctions, it requires:

[Plan Length=k-1]

With Local Sensor:

The sensor identifies the actual starting location.

The robot can then choose a route specifically for that location.

Average plan length:

[Average=frac{0+1+2+cdots+(k-1)}{k}]

[boxed{Average=frac{k-1}{2}}]

The maximum plan length is:

[boxed{Maximum=k-1}]

The experiment demonstrates that local sensing reduces unnecessary movement on average.

Note: This is a simplified deterministic corridor experiment. The sensor is assumed to identify the junction correctly, and each LEFT action moves exactly one junction towards the pharmacy. The belief-state counts are for this particular model, not the full (2^V) hospital belief-state space.

12.12 Final Defence

The original design must be modified because it assumes that the robot knows its exact starting location.

Belief-state search allows the robot to represent multiple possible positions, while local sensing helps identify the actual position.

As the number of possible locations increases, maintaining uncertainty becomes more difficult. Local sensing reduces this uncertainty and helps the robot choose an appropriate route.

Final Decision: MODIFY the original UCS-based design by adding belief-state representation and local sensing.

This provides a more reliable solution when the robot's localisation sensor fails.

Overall Conclusion

The assignment investigates three uninformed search algorithms for a hospital medicine-delivery robot: BFS, recursive DFS and UCS.

The mathematical analysis explains their time and space requirements. The Python implementation provides a way to compare their performance for hospital graphs containing 1,000 to 1,000,000 states.

The comparison shows that each algorithm has different advantages. BFS finds minimum-edge routes, DFS uses less frontier memory, and UCS finds minimum-travel-cost routes.

Finally, the Day-4 challenge demonstrates that the original design must be modified when the localisation sensor fails. Belief-state representation and local sensing help the robot handle multiple possible positions and reach the pharmacy safely.

Overall, UCS with belief-state representation and local sensing is recommended for the hospital medicine-delivery robot.